

Python's functools Module

Implementing a cache by adding a global variable and code to manage it for every function you'd like to memoize can be quite a chore. Python's standard library has a `functools` module with a function decorator named `@lru_cache()` that automatically memoizes the function it decorates. In Python syntax, this means adding `@lru_cache()` to the line preceding the function's `def` statement.

The cache can have a memory size limit set. The *lru* in the decorator name stands for the *least recently used* cache replacement policy, meaning that the least recently used entry is replaced with new entries when the cache limit is reached. The LRU algorithm is simple and fast, though other cache replacement policies are available for different software requirements.

The *fibonacciFunctools.py* program demonstrates the use of the `@lru_cache()` decorator. The additions to the original *fibonacciByRecursion.py* program from Chapter 2 have been marked in bold:

Python

```
import functools

@functools.lru_cache()
def fibonacci(nthNumber):
    print('fibonacci(%s) called.' % (nthNumber))
    if nthNumber == 1 or nthNumber == 2:
        # BASE CASE
        print('Call to fibonacci(%s) returning 1.' % (nthNumber))
        return 1
    else:
        # RECURSIVE CASE
        print('Calling fibonacci(%s) (nthNumber - 1).' % (nthNumber - 1))
        result = fibonacci(nthNumber - 1)

        print('Calling fibonacci(%s) (nthNumber - 2).' % (nthNumber - 2))
        result = result + fibonacci(nthNumber - 2)

    print('Call to fibonacci(%s) returning %s.' % (nthNumber, result))
```